

Proyecto Final del Curso

Sistema de Gestión de Biblioteca Personal con Interfaz Gráfica

Curso de Programación en Python

Tiempo de entrega: 2 semanas

1. Introducción

¡Felicidades por llegar al proyecto final! Este proyecto integra **todos los conceptos** que has aprendido durante el curso. Vas a crear una aplicación completa y profesional: un **Sistema de Gestión de Biblioteca Personal** con interfaz gráfica usando `tkinter`.

Este proyecto no solo demuestra tus habilidades técnicas, sino también tu capacidad para integrar múltiples conceptos, diseñar software robusto y crear aplicaciones con valor práctico real.

2. Descripción General del Proyecto

Crearás un sistema completo para gestionar una biblioteca personal de libros. La aplicación permitirá:

- Agregar, buscar, editar y eliminar libros
- Categorizar libros por género, autor, año, etc.
- Registrar préstamos a amigos/familiares
- Generar estadísticas sobre la colección
- Importar/exportar datos desde/hacia archivos CSV
- Validar datos de entrada
- Buscar libros usando expresiones regulares
- Guardar y recuperar datos automáticamente
- Interfaz gráfica intuitiva y profesional

3. Objetivos de Aprendizaje

Al completar este proyecto, demostrarás dominio de:

3.1. Conceptos Fundamentales (Semanas 0-2)

- Variables y tipos de datos
- Funciones y parámetros
- Condicionales complejos
- Bucles y validación iterativa
- Formateo de strings con f-strings

3.2. Conceptos Intermedios (Semanas 3-5)

- Manejo robusto de excepciones
- Uso de librerías estándar y externas (`datetime`, `csv`, `re`, `statistics`, etc.)
- Pruebas unitarias con `pytest`
- Validación y sanitización de datos

3.3. Conceptos Avanzados (Semanas 6-9)

- Lectura y escritura de archivos
- Trabajo con CSV
- Expresiones regulares para búsqueda y validación
- Programación orientada a objetos (clases, herencia, encapsulación)
- Comprehensions (list, dict)
- Funciones lambda
- Decoradores
- Interfaz gráfica con `tkinter`

4. Requisitos Funcionales

4.1. Módulo 1: Clase Libro (POO)

Crea una clase `Libro` con los siguientes atributos y métodos:

Listing 1: Estructura básica de la clase `Libro`

```
1 class Libro:
2     def __init__(self, isbn, titulo, autor, genero, anio_publicacion):
3         # Atributos privados con validacion
4         self._isbn = isbn
5         self._titulo = titulo
6         self._autor = autor
7         self._genero = genero
8         self._anio_publicacion = anio_publicacion
9         self._prestado = False
```

```
10         self._prestado_a = None
11         self._fecha_prestamo = None
12
13     # Properties con @property
14     @property
15     def isbn(self):
16         return self._isbn
17
18     # Metodos de validacion
19     @staticmethod
20     def validar_isbn(isbn):
21         # Usar regex para validar formato ISBN
22         pass
23
24     # Metodos especiales
25     def __str__(self):
26         pass
27
28     def __repr__(self):
29         pass
30
31     # Metodos de negocio
32     def prestar(self, nombre_persona):
33         pass
34
35     def devolver(self):
36         pass
```

Requisitos de la clase:

- Validar ISBN usando expresiones regulares (formato ISBN-10 o ISBN-13)
- Validar que el año de publicación sea razonable (entre 1000 y año actual)
- Usar `@property` para todos los atributos principales
- Método `__str__()` que retorne representación legible
- Método `__repr__()` para debugging
- Método para prestar libro (registra fecha usando `datetime`)
- Método para devolver libro
- Al menos un método de clase (`@classmethod`) y uno estático (`@staticmethod`)

4.2. Módulo 2: Clase Biblioteca (POO y Gestión)

Crea una clase `Biblioteca` que gestione la colección:

Listing 2: Estructura de la clase `Biblioteca`

```
1 class Biblioteca:
2     def __init__(self, nombre="Mi Biblioteca"):
3         self._nombre = nombre
4         self._libros = {} # Diccionario: ISBN -> Libro
```

```
5         self._archivo_datos = "biblioteca.csv"
6         self.cargar_desde_archivo()
7
8     def agregar_libro(self, libro):
9         # Agregar con validacion de duplicados
10        pass
11
12    def buscar_por_titulo(self, patron):
13        # Búsqueda con regex
14        pass
15
16    def buscar_por_autor(self, patron):
17        # Búsqueda con regex
18        pass
19
20    def eliminar_libro(self, isbn):
21        # Eliminar con confirmacion
22        pass
23
24    def listar_libros(self, filtro=None):
25        # Retorna lista filtrada
26        pass
27
28    def obtener_estadisticas(self):
29        # Usar statistics para calcular promedios, etc.
30        pass
31
32    def guardar_a_archivo(self):
33        # Guardar en CSV
34        pass
35
36    def cargar_desde_archivo(self):
37        # Cargar desde CSV con manejo de excepciones
38        pass
39
40    def exportar_csv(self, archivo):
41        # Exportar a CSV personalizado
42        pass
43
44    def importar_csv(self, archivo):
45        # Importar desde CSV con validacion
46        pass
```

Requisitos de la clase:

- Usar diccionario para almacenar libros (clave: ISBN)
- Búsqueda con expresiones regulares (insensible a mayúsculas)
- Guardar automáticamente después de cada cambio
- Manejo robusto de excepciones en I/O de archivos
- Método que use `statistics` para calcular estadísticas (año promedio, etc.)

- Usar comprehensions para filtrar y transformar datos
- Funciones lambda en operaciones de ordenamiento y filtrado

4.3. Módulo 3: Interfaz Gráfica (Tkinter)

Crea una interfaz gráfica completa con las siguientes ventanas/secciones:

4.3.1. Ventana Principal

- Menú superior con opciones: Archivo, Gestión, Estadísticas, Ayuda
- Panel lateral con botones para funciones principales
- Área central con tabla/lista de libros
- Barra de estado inferior

4.3.2. Ventana de Agregar/Editar Libro

- Campos de entrada para todos los atributos del libro
- Validación en tiempo real
- Mensajes de error claros
- Botones Guardar y Cancelar

4.3.3. Ventana de Búsqueda

- Campo de búsqueda con opciones (por título, autor, género)
- Checkbox para usar expresiones regulares
- Lista de resultados
- Botones para acciones sobre resultados

4.3.4. Ventana de Estadísticas

- Total de libros en la colección
- Libros por género (gráfico o tabla)
- Año promedio de publicación
- Libros actualmente prestados
- Top 5 autores con más libros

4.3.5. Ventana de Préstamos

- Registrar nuevo préstamo
- Ver préstamos activos
- Devolver libro
- Historial de préstamos

4.4. Módulo 4: Validaciones y Excepciones

Implementa manejo robusto de errores:

- **Validación de ISBN:** Regex para ISBN-10 e ISBN-13
- **Validación de año:** Debe estar entre 1000 y año actual
- **Validación de campos vacíos:** Ningún campo puede estar vacío
- **Manejo de archivos:** try-except-finally para I/O
- **Duplicados:** Verificar ISBN duplicado antes de agregar
- **Excepciones personalizadas:** Crea al menos 2 excepciones custom

Ejemplo de excepción personalizada:

Listing 3: Excepciones personalizadas

```
1 class ISBNInvalidoError(Exception):
2     """Excepcion para ISBN invalido"""
3     pass
4
5 class LibroDuplicadoError(Exception):
6     """Excepcion cuando se intenta agregar libro duplicado"""
7     pass
8
9 class LibroNoEncontradoError(Exception):
10     """Excepcion cuando no se encuentra un libro"""
11     pass
```

4.5. Módulo 5: Archivos y Persistencia

- **Archivo principal:** biblioteca.csv para guardar todos los libros
- **Auto-guardado:** Guardar automáticamente después de cada cambio
- **Carga al inicio:** Cargar datos existentes al iniciar la aplicación
- **Exportar:** Permitir exportar a CSV personalizado
- **Importar:** Permitir importar desde CSV con validación
- **Backup:** Crear copia de seguridad antes de importar
- **Formato CSV:** ISBN, Título, Autor, Género, Año, Prestado, Prestado_a, Fecha_Prestamo

4.6. Módulo 6: Expresiones Regulares

Usa regex para:

- **Validar ISBN:**
 - ISBN-10: `^(?:\d{9}X|\d{10})$`
 - ISBN-13: `^\d{13}$`

- **Búsqueda flexible:** Permitir búsqueda parcial insensible a mayúsculas
- **Validar año:** Solo dígitos, 4 caracteres
- **Extraer información:** De strings complejos durante importación

4.7. Módulo 7: Pruebas Unitarias

Crea pruebas con `pytest` para:

- **Clase Libro:**
 - Test de creación de libro válido
 - Test de ISBN inválido (debe lanzar excepción)
 - Test de año inválido
 - Test de préstamo y devolución
 - Test de métodos `__str__` y `__repr__`
- **Clase Biblioteca:**
 - Test de agregar libro
 - Test de agregar libro duplicado (debe lanzar excepción)
 - Test de búsqueda por título
 - Test de búsqueda por autor con regex
 - Test de eliminar libro
 - Test de estadísticas
 - Test de guardar y cargar desde archivo
- **Validaciones:**
 - Test de validación de ISBN (casos válidos e inválidos)
 - Test de validación de año
 - Test de campos vacíos

Estructura de archivos de prueba:

Listing 4: Estructura de pruebas

```
1 tests/  
2     __init__.py  
3     test_libro.py  
4     test_biblioteca.py  
5     test_validaciones.py  
6     conftest.py  # Fixtures compartidas
```

5. Requisitos Técnicos Detallados

5.1. Estructura del Proyecto

Tu proyecto debe tener la siguiente estructura:

Listing 5: Estructura de directorios

```
1 biblioteca_personal/  
2     main.py                # Punto de entrada  
3     libro.py               # Clase Libro  
4     biblioteca.py          # Clase Biblioteca  
5     gui.py                 # Interfaz grafica  
6     validaciones.py        # Funciones de validacion  
7     excepciones.py         # Excepciones personalizadas  
8     utilidades.py          # Funciones auxiliares  
9     config.py              # Configuracion  
10    requirements.txt        # Dependencias  
11    README.md               # Documentacion  
12    tests/                  # Pruebas unitarias  
13        __init__.py  
14        test_libro.py  
15        test_biblioteca.py  
16        test_validaciones.py  
17        conftest.py  
18    datos/                  # Archivos de datos  
19        biblioteca.csv  
20        backup/
```

5.2. Uso de Conceptos por Módulo

5.2.1. Variables y Tipos de Datos (Semana 0)

- Usar f-strings para todo el formateo
- Variables descriptivas en todo el código
- Conversión de tipos cuando sea necesario
- Métodos de strings: `strip()`, `title()`, `upper()`, `lower()`

5.2.2. Condicionales (Semana 1)

- Validaciones complejas con múltiples condiciones
- Uso de operadores lógicos (`and`, `or`, `not`)
- Match-case para menús o categorización (Python 3.10+)

5.2.3. Bucles (Semana 2)

- Iterar sobre colecciones de libros
- Validación de entrada con bucles `while`
- Uso de `break` y `continue`
- Bucles `for` con `range()` y sobre iterables

5.2.4. Excepciones (Semana 3)

- Bloques `try-except-else-finally` en I/O de archivos
- Excepciones personalizadas heredando de `Exception`
- Uso de `raise` para lanzar excepciones
- Manejo específico de excepciones (`FileNotFoundError`, `ValueError`, etc.)

5.2.5. Librerías (Semana 4)

- `tkinter`: Interfaz gráfica completa
- `csv`: Lectura y escritura de archivos
- `datetime`: Registro de fechas de préstamo
- `re`: Expresiones regulares para búsqueda y validación
- `statistics`: Cálculo de estadísticas
- `os`: Manejo de rutas y archivos
- `sys`: Información del sistema

5.2.6. Pruebas Unitarias (Semana 5)

- Mínimo 20 tests con `pytest`
- Uso de `assert` para verificaciones
- Fixtures con `@pytest.fixture`
- Tests de excepciones con `pytest.raises`
- Cobertura de código ¿80 %

5.2.7. Archivos (Semana 6)

- Lectura y escritura de CSV con módulo `csv`
- Uso de `with` para manejo automático de archivos
- Creación de backups automáticos
- Manejo de rutas con `os.path`

5.2.8. Expresiones Regulares (Semana 7)

- `re.match()` para validar ISBN
- `re.search()` para búsquedas en títulos/autores
- `re.findall()` para extraer información
- Flags: `re.IGNORECASE`

5.2.9. POO (Semana 8)

- Mínimo 2 clases principales: Libro y Biblioteca
- Encapsulación con atributos privados (`_atributo`)
- Properties con `@property`
- Métodos de clase con `@classmethod`
- Métodos estáticos con `@staticmethod`
- Métodos especiales: `__init__`, `__str__`, `__repr__`
- Bonus: Herencia (crear subclases de Libro: LibroFisico, LibroDigital)

5.2.10. Conceptos Avanzados (Semana 9)

- List comprehensions para filtrar y transformar listas
- Dict comprehensions para crear diccionarios
- Funciones lambda en `sort()`, `filter()`, `map()`
- Sets para operaciones de conjuntos (géneros únicos, etc.)
- `*args` y `**kwargs` en funciones flexibles
- Decorador personalizado (ej: `@log_operacion`)

6. Ejemplo de Código: Integración de Conceptos

Listing 6: Ejemplo de método que integra múltiples conceptos

```
1 import re
2 import csv
3 from datetime import datetime
4 from typing import List, Dict, Optional
5
6 class Biblioteca:
7     def buscar_libros_avanzado(
8         self,
9         patron: str,
10        campo: str = "titulo",
11        usar_regex: bool = True,
12        **filtros
13    ) -> List[Dict]:
14        """
15        Búsqueda avanzada de libros.
16
17        Integra: regex, excepciones, comprehensions,
18        *args/**kwargs, type hints
19        """
20        try:
21            # Validar campo de búsqueda
22            campos_validos = {"titulo", "autor", "genero"}
```

```
23         if campo not in campos_validos:
24             raise ValueError(
25                 f"Campo debe ser uno de: {campos_validos}"
26             )
27
28         # Preparar patron de busqueda
29         if usar_regex:
30             patron_compilado = re.compile(
31                 patron,
32                 re.IGNORECASE
33             )
34         else:
35             patron = patron.lower()
36
37         # Buscar usando comprehension
38         resultados = [
39             libro for libro in self._libros.values()
40             if self._coincide(
41                 libro,
42                 campo,
43                 patron,
44                 patron_compilado if usar_regex else None
45             )
46         ]
47
48         # Aplicar filtros adicionales
49         if filtros:
50             resultados = self._aplicar_filtros(
51                 resultados,
52                 **filtros
53             )
54
55         # Ordenar usando lambda
56         resultados.sort(
57             key=lambda libro: getattr(libro, campo)
58         )
59
60         return [libro.to_dict() for libro in resultados]
61
62     except re.error as e:
63         raise ValueError(
64             f"Expresion regular invalida: {e}"
65         )
66     except Exception as e:
67         raise RuntimeError(
68             f"Error en busqueda: {e}"
69         )
```

7. Interfaz Gráfica con Tkinter

7.1. Ventana Principal - Ejemplo

Listing 7: Estructura básica de la interfaz

```
1 import tkinter as tk
2 from tkinter import ttk, messagebox
3
4 class BibliotecaGUI:
5     def __init__(self, biblioteca):
6         self.biblioteca = biblioteca
7         self.ventana = tk.Tk()
8         self.ventana.title("Sistema de Biblioteca Personal")
9         self.ventana.geometry("1000x600")
10
11         self._crear_menu()
12         self._crear_barra_herramientas()
13         self._crear_tabla_libros()
14         self._crear_panel_detalle()
15         self._crear_barra_estado()
16
17         self.actualizar_lista()
18
19     def _crear_menu(self):
20         menubar = tk.Menu(self.ventana)
21         self.ventana.config(menu=menubar)
22
23         # Menu Archivo
24         menu_archivo = tk.Menu(menubar, tearoff=0)
25         menubar.add_cascade(label="Archivo", menu=menu_archivo)
26         menu_archivo.add_command(
27             label="Importar CSV",
28             command=self.importar_csv
29         )
30         menu_archivo.add_command(
31             label="Exportar CSV",
32             command=self.exportar_csv
33         )
34         menu_archivo.add_separator()
35         menu_archivo.add_command(
36             label="Salir",
37             command=self.ventana.quit
38         )
39
40         # Menu Gestion
41         menu_gestion = tk.Menu(menubar, tearoff=0)
42         menubar.add_cascade(label="Gestion", menu=menu_gestion)
43         menu_gestion.add_command(
44             label="Agregar Libro",
45             command=self.agregar_libro
46         )
47         menu_gestion.add_command(
48             label="Editar Libro",
49             command=self.editar_libro
50         )
51         menu_gestion.add_command(
52             label="Eliminar Libro",
```

```
53         command=self.eliminar_libro
54     )
55
56     # Menu Estadísticas
57     menu_stats = tk.Menu(menu_bar, tearoff=0)
58     menu_bar.add_cascade(
59         label="Estadísticas",
60         menu=menu_stats
61     )
62     menu_stats.add_command(
63         label="Ver Estadísticas",
64         command=self.mostrar_estadísticas
65     )
66
67 def _crear_tabla_libros(self):
68     # Frame para la tabla
69     frame_tabla = ttk.Frame(self.ventana)
70     frame_tabla.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
71
72     # Scrollbars
73     scroll_y = ttk.Scrollbar(frame_tabla, orient=tk.VERTICAL)
74     scroll_x = ttk.Scrollbar(frame_tabla, orient=tk.HORIZONTAL)
75
76     # Treeview
77     self.tabla = ttk.Treeview(
78         frame_tabla,
79         columns=("ISBN", "Titulo", "Autor", "Genero", "Anio"),
80         show="headings",
81         yscrollcommand=scroll_y.set,
82         xscrollcommand=scroll_x.set
83     )
84
85     # Configurar columnas
86     self.tabla.heading("ISBN", text="ISBN")
87     self.tabla.heading("Titulo", text="Titulo")
88     self.tabla.heading("Autor", text="Autor")
89     self.tabla.heading("Genero", text="Genero")
90     self.tabla.heading("Anio", text="Anio")
91
92     self.tabla.column("ISBN", width=120)
93     self.tabla.column("Titulo", width=300)
94     self.tabla.column("Autor", width=200)
95     self.tabla.column("Genero", width=100)
96     self.tabla.column("Anio", width=80)
97
98     # Empaquetar
99     scroll_y.config(command=self.tabla.yview)
100    scroll_x.config(command=self.tabla.xview)
101    scroll_y.pack(side=tk.RIGHT, fill=tk.Y)
102    scroll_x.pack(side=tk.BOTTOM, fill=tk.X)
103    self.tabla.pack(fill=tk.BOTH, expand=True)
104
105    # Doble click para editar
```

```
106 self.tabla.bind("<Double-1>", lambda e: self.editar_libro())
```

7.2. Validación en Tiempo Real

Listing 8: Validación de ISBN en tiempo real

```
1 def _validar_isbn_tiempo_real(self, event):
2     """
3     Valida ISBN mientras el usuario escribe.
4     Integra: regex, tkinter, excepciones
5     """
6     isbn = self.entry_isbn.get().strip()
7
8     if not isbn:
9         self.label_isbn_estado.config(
10             text="",
11             foreground="black"
12         )
13         return
14
15     try:
16         if Libro.validar_isbn(isbn):
17             self.label_isbn_estado.config(
18                 text="ISBN valido",
19                 foreground="green"
20             )
21             self.btn_guardar.config(state=tk.NORMAL)
22         else:
23             raise ISBNInvalidoError("Formato incorrecto")
24     except ISBNInvalidoError as e:
25         self.label_isbn_estado.config(
26             text=f"ISBN invalido: {e}",
27             foreground="red"
28         )
29         self.btn_guardar.config(state=tk.DISABLED)
```

8. Funcionalidades Adicionales Requeridas

8.1. Sistema de Préstamos

- Registrar préstamo con nombre de persona y fecha
- Calcular días transcurridos desde el préstamo
- Alerta si un libro lleva más de 30 días prestado
- Historial de préstamos anteriores

8.2. Estadísticas Avanzadas

- Total de libros por género (usar `collections.Counter`)

- Año promedio de publicación (usar `statistics.mean`)
- Top 5 autores con más libros
- Porcentaje de libros prestados vs. disponibles
- Libro más antiguo y más reciente

8.3. Búsqueda y Filtrado

- Búsqueda simple (substring)
- Búsqueda con regex
- Filtro por rango de años
- Filtro por género
- Filtro por estado (prestado/disponible)
- Combinar múltiples filtros

8.4. Importación/Exportación

- Exportar toda la biblioteca a CSV
- Exportar solo libros filtrados
- Importar desde CSV con validación
- Crear backup automático antes de importar
- Reporte de errores durante importación

9. Características de Calidad

9.1. Documentación

- **Docstrings:** Todas las clases y funciones públicas deben tener docstrings
- **Type hints:** Usar anotaciones de tipo en todas las funciones
- **README.md:** Documentación completa del proyecto
- **Comentarios:** Explicar lógica compleja

9.2. Código Limpio

- Nombres descriptivos para variables, funciones y clases
- Funciones pequeñas y enfocadas (máx. 30-40 líneas)
- Evitar código duplicado (DRY - Don't Repeat Yourself)
- Indentación consistente (4 espacios)
- Líneas de máximo 79-100 caracteres

9.3. Manejo de Errores

- Nunca dejar bloques `except` vacíos
- Mensajes de error claros y útiles
- Logging de errores para debugging
- Validación de entrada del usuario

10. README.md Requerido

Tu archivo README.md debe incluir:

Listing 9: Contenido del README

```
1 # Sistema de Gestion de Biblioteca Personal
2
3 ## Descripcion
4 [Descripcion breve del proyecto]
5
6 ## Caracteristicas
7 - Lista de funcionalidades principales
8
9 ## Requisitos
10 - Python 3.10+
11 - Librerias: tkinter, pytest, etc.
12
13 ## Instalacion
14 '''bash
15 pip install -r requirements.txt
16 '''
17
18 ## Uso
19 '''bash
20 python main.py
21 '''
22
23 ## Estructura del Proyecto
24 [Descripcion de archivos y carpetas]
25
26 ## Pruebas
27 '''bash
28 pytest tests/ -v
29 pytest --cov=. tests/ # Con cobertura
30 '''
31
32 ## Conceptos de Python Utilizados
33 [Lista de todos los conceptos integrados]
34
35 ## Autor
36 [Tu nombre]
37
38 ## Fecha
39 [Fecha de entrega]
```

11. Entregables

11.1. Código Fuente

1. Todos los archivos `.py` mencionados en la estructura
2. Archivo `requirements.txt` con dependencias
3. Archivo `README.md` completo
4. Carpeta `tests/` con todas las pruebas
5. Archivo CSV de ejemplo con datos de prueba

11.2. Documentación

1. `README.md` completo
2. Docstrings en todas las clases y funciones públicas
3. Comentarios explicativos en lógica compleja
4. Manual de usuario (opcional, puntos extra)

11.3. Pruebas

1. Mínimo 20 pruebas unitarias
2. Cobertura de código ¿80 %
3. Todas las pruebas deben pasar
4. Incluir tests de excepciones

12. Defensa del Proyecto (60 % de la nota final)

12.1. Formato de la Defensa

La defensa es **obligatoria** y consta de:

1. **Presentación (10 minutos):**
 - Demo completa de la aplicación funcionando
 - Mostrar todas las funcionalidades implementadas
 - Ejecutar suite de pruebas
2. **Explicación del código (15 minutos):**
 - Arquitectura general del proyecto
 - Explicación de clases principales
 - Decisiones de diseño importantes
 - Cómo integraste cada concepto del curso
3. **Preguntas técnicas (15 minutos):**

- Explicar partes específicas del código
- Modificar código en vivo
- Resolver bugs introducidos intencionalmente
- Agregar funcionalidad pequeña

12.2. Preguntas Tipo de la Defensa

Prepárate para responder preguntas como:

- “Explica cómo funciona tu validación de ISBN”
- “¿Qué pasa si cambias este `float()` por `int()`?”
- “¿Por qué usaste un diccionario en lugar de una lista?”
- “Muestra dónde usaste list comprehensions y explica por qué”
- “¿Cómo manejas el caso de un archivo CSV corrupto?”
- “Explica esta expresión regular”
- “¿Qué hace este decorador?”
- “Modifica esta búsqueda para que sea case-sensitive”
- “Agrega un nuevo campo a la clase Libro”

12.3. Demostración de Comprensión

Durante la defensa, debes demostrar que:

- Entiendes cada línea de código que escribiste
- Puedes explicar por qué elegiste cada solución
- Conoces alternativas y sus trade-offs
- Puedes modificar el código en tiempo real
- Comprendes cómo los conceptos se integran

13. Rúbrica de Evaluación

13.1. Distribución General

- **Implementación (40 %):** 40 puntos
- **Defensa (60 %):** 60 puntos
- **Total:** 100 puntos
- **Puntos Extra:** Hasta +20 puntos

13.2. Parte 1: Implementación (40 puntos)

13.2.1. 1.1 Funcionalidad Core (16 puntos)

- **CRUD completo (8 pts):** Crear, leer, actualizar, eliminar libros funciona perfectamente
- **Persistencia (4 pts):** Guarda y carga datos correctamente
- **Búsqueda (4 pts):** Búsqueda funcional con regex

13.2.2. 1.2 Programación Orientada a Objetos (8 puntos)

- **Diseño de clases (3 pts):** Clases bien diseñadas y cohesivas
- **Encapsulación (2 pts):** Uso correcto de atributos privados y properties
- **Métodos especiales (2 pts):** `__init__`, `__str__`, `__repr__`
- **Métodos de clase/estáticos (1 pt):** Al menos uno de cada tipo

13.2.3. 1.3 Manejo de Errores y Validaciones (6 puntos)

- **Excepciones personalizadas (2 pts):** Al menos 2, bien implementadas
- **Validaciones robustas (3 pts):** ISBN, año, campos vacíos, etc.
- **Manejo de I/O (1 pt):** Try-except apropiado en archivos

13.2.4. 1.4 Interfaz Gráfica (6 puntos)

- **Diseño funcional (3 pts):** Interfaz completa y usable
- **Validación en tiempo real (2 pts):** Feedback inmediato
- **UX (1 pt):** Fácil de usar, mensajes claros

13.2.5. 1.5 Calidad de Código (4 puntos)

- **Documentación (2 pts):** Docstrings, comentarios, README
- **Organización (1 pt):** Estructura de archivos clara
- **Estilo (1 pt):** Código limpio y legible

13.3. Parte 2: Defensa del Proyecto (60 puntos)

13.3.1. 2.1 Demostración (18 puntos)

- **Funcionalidad completa (10 pts):** Demuestra todas las funciones
- **Manejo de casos (5 pts):** Muestra casos normales y edge cases
- **Pruebas (3 pts):** Ejecuta suite de pruebas exitosamente

13.3.2. 2.2 Explicación Técnica (24 puntos)

- **Arquitectura (6 pts):** Explica estructura general y flujo
- **POO (6 pts):** Explica diseño de clases y decisiones
- **Integración de conceptos (6 pts):** Muestra cómo usó cada tema del curso
- **Decisiones de diseño (6 pts):** Justifica elecciones técnicas

13.3.3. 2.3 Respuestas a Preguntas (18 puntos)

- **Comprensión del código (10 pts):** Explica cualquier parte solicitada
- **Modificación en vivo (5 pts):** Puede cambiar código efectivamente
- **Resolución de problemas (3 pts):** Debugging y solución de bugs

13.4. Parte 3: Pruebas Unitarias (Requisito, no puntuado separadamente)

Mínimo requerido:

- 20 pruebas unitarias
- Cobertura ¿80 %
- Todas las pruebas pasan

Nota: Si no cumples los requisitos mínimos de pruebas, se penaliza la sección de implementación.

13.5. Puntos Extra (Hasta +20 puntos)**13.5.1. Funcionalidades Avanzadas (+10 pts)**

- **Herencia (+3 pts):** Subclases de Libro (LibroFisico, LibroDigital)
- **Gráficos (+3 pts):** Visualización de estadísticas con matplotlib
- **Base de datos (+4 pts):** Usar SQLite en lugar de CSV

13.5.2. Calidad Excepcional (+10 pts)

- **Documentación completa (+3 pts):** Manual de usuario detallado
- **Cobertura ¿95 % (+2 pts):** Pruebas exhaustivas
- **Temas (+2 pts):** Interfaz con temas personalizables
- **Logging (+3 pts):** Sistema de logs completo

13.6. Penalizaciones

- **-100 % defensa:** No asiste a defensa (máx. 40/100)
- **-50 % defensa:** No puede explicar código (posible plagio)
- **-10 pts:** Menos de 20 pruebas unitarias
- **-10 pts:** Cobertura ¡80 %
- **-5 pts:** Sin README.md o incompleto
- **-5 pts:** Estructura de proyecto incorrecta
- **-3 pts por día:** Entrega tardía (máx. 3 días)

13.7. Nota Mínima Aprobatoria

Para aprobar el proyecto final necesitas obtener al menos **70 puntos de 100**.

13.8. Escala de Calificación

- **90-100+ pts:** Excelente - Proyecto profesional
- **80-89 pts:** Muy Bueno - Dominio sólido
- **70-79 pts:** Bueno - Competente
- **60-69 pts:** Suficiente - Necesita mejorar
- **0-59 pts:** Insuficiente - No aprobado

14. Cronograma Sugerido

14.1. Semana 1

- Diseñar arquitectura del proyecto
- Implementar clases Libro y Biblioteca
- Crear funciones de validación
- Escribir primeras pruebas unitarias

14.2. Semana 2

- Implementar interfaz gráfica básica
- Agregar funcionalidad CRUD
- Implementar persistencia con CSV
- Completar pruebas unitarias
- Escribir documentación
- Preparar defensa

15. Recursos de Apoyo

15.1. Documentación Oficial

- Python: <https://docs.python.org/3/>
- Tkinter: <https://docs.python.org/3/library/tkinter.html>
- Pytest: <https://docs.pytest.org/>
- CSV: <https://docs.python.org/3/library/csv.html>
- Regex: <https://docs.python.org/3/library/re.html>

15.2. Tutoriales Recomendados

- Real Python - Tkinter GUI Programming
- Corey Schafer - OOP en Python (YouTube)
- Python Testing with pytest (Brian Okken)

16. Preguntas Frecuentes

P: ¿Puedo usar una base de datos en lugar de CSV?

R: Sí, pero CSV es requisito mínimo. Si usas BD, también debes mantener export/import CSV y obtienes puntos extra.

P: ¿Cuántas líneas de código debería tener?

R: No hay mínimo/máximo, pero típicamente 800-1500 líneas para un proyecto completo (incluyendo tests).

P: ¿Puedo trabajar en equipo?

R: No, este es un proyecto individual. Debes poder explicar cada línea.

P: ¿Qué pasa si no termino todo?

R: Entrega lo que tengas. Es mejor entregar algo funcional parcial que nada. La defensa determinará tu comprensión.

P: ¿Puedo usar librerías adicionales?

R: Sí, pero las principales funcionalidades deben usar las librerías del curso. Librerías adicionales son bonus.

P: ¿Cómo sé si mi interfaz es suficientemente buena?

R: Debe ser funcional, no necesariamente bonita. Si un usuario puede realizar todas las operaciones sin confusión, es suficiente.

17. Consejos Finales

- **Empieza temprano:** Este proyecto es complejo y toma tiempo.
- **Commits frecuentes:** Usa Git para versionar tu código.
- **Prueba constantemente:** No esperes al final para probar.
- **Documenta mientras programas:** Es más fácil que hacerlo al final.

- **Pide ayuda:** Si te trabas, consulta con el profesor o compañeros (pero escribe tu propio código).
- **Prioriza:** Primero implementa lo básico, luego añade extras.
- **Practica tu defensa:** Explica tu código en voz alta antes de la reunión.
- **Ten backup:** Guarda tu código en la nube (GitHub, Drive, etc.).
- **Lee los errores:** Python te dice exactamente qué está mal.
- **Diviértete:** Este es tu proyecto, hazlo a tu manera dentro de los requisitos.

¡Éxito en tu proyecto final!

*Este proyecto demuestra todo lo que has aprendido.
Es tu oportunidad de brillar y mostrar tus habilidades como programador.
Confía en ti mismo y en todo lo que has aprendido durante el curso.*

¡Adelante!